

# GROOT

GROWTH | REASONING | OBSERVATION | ORCHESTRATION | THINKING

## ORGANIZATIONAL INTELLIGENCE THROUGH NATURAL LANGUAGE

Final Concept Paper | Track 01

### ONE-SENTENCE THESIS

GROOT turns natural human language - typed explicitly or spoken naturally in a decision-making environment - into executable intelligence missions that can answer questions, assemble decision-ready information, and learn from organizational outcomes over time.

Prepared as a competition-ready product and technical concept

# GROOT in one page

GROOT is not a scraping platform, a dashboard, or a chatbot wrapper. Its primary interface is natural language. A user can type or speak a requirement such as “Should we switch suppliers?”, “Why did sales fall in the North?”, or “Find 30 manufacturers matching these constraints.” GROOT interprets the intent, determines what information and workflow are required, and executes the appropriate intelligence mission.

The second input path is ambient: with explicit consent, GROOT can sit inside a decision room and listen to natural conversation. It does not treat the meeting as a transcription problem. It detects decision-relevant statements, assumptions, objections, unresolved information gaps and emerging questions, and turns them into the same internal mission structure generated from an explicit prompt.

The long-term system is a persistent decision-learning layer. It records what was known, what was assumed, what was missing, what decision was made, and what happened afterward. Over repeated decisions, GROOT can update the organization-specific decision model - not by continuously retraining a base LLM, but by maintaining a growing decision memory and updating the variables, evidence paths and reasoning patterns that it uses.

| Core layer                                                                                                                                                                                                                           | What GROOT does                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Natural-language interface                                                                                                                                                                                                           | Accepts a plain-English prompt or naturally occurring spoken language.                 |
| Mission compiler                                                                                                                                                                                                                     | Converts language into the required task, data requirements, constraints and workflow. |
| Context engine                                                                                                                                                                                                                       | Combines permitted internal data, files, meeting context and external sources.         |
| Decision graph                                                                                                                                                                                                                       | Represents evidence, unknowns, assumptions, alternatives and dependencies.             |
| Data acquisition                                                                                                                                                                                                                     | Obtains only the information needed to complete the mission.                           |
| Decision state                                                                                                                                                                                                                       | Maintains the current state of an open or completed organizational decision.           |
| Learning engine                                                                                                                                                                                                                      | Compares decisions with outcomes and updates organization-specific decision memory.    |
| THE CENTRAL IDEA                                                                                                                                                                                                                     |                                                                                        |
| The user does not need to know what data, columns, APIs or workflow steps are required. The user states what they need to know, understand, investigate or decide. GROOT determines the information architecture behind the request. |                                                                                        |

# The gap is not access to AI. It is the gap between language and executable intelligence.

Modern AI assistants can already research multi-step questions and work across connected information. OpenAI documents Deep Research as a system that can plan, research and synthesize complex questions using web, files and connected apps. Anthropic similarly offers enterprise context, connectors and tool access. Microsoft has gone further with Work IQ, a workplace intelligence layer that continuously builds semantic understanding across email, meetings, files, people and business systems. [1][2][3]

That means GROOT cannot claim novelty simply because it can understand natural language, listen to meetings, or connect to company information. Those capabilities are increasingly becoming platform primitives. GROOT must differentiate at the level of the system it builds around those capabilities.

## The actual unmet need

- Organizations express requirements in human language, while the data needed to fulfill those requirements is fragmented across systems, documents, conversations and external information.
- The user often knows the outcome they want, but does not know the complete data specification needed to produce it.
- In a live meeting, the information requirement may be implicit: people discuss an issue, reveal an uncertainty, or raise an objection without formally creating a data request.
- A useful answer is not always the same thing as a useful decision state. Decisions need assumptions, alternatives, unresolved variables, evidence and later outcome tracking.

| THE DESIGN QUESTION                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Can one system convert ordinary human language - including language spoken inside a meeting - into a machine-executable intelligence mission, then preserve the resulting decision state and learn from what actually happened? |

## Why “just build a better chatbot” is insufficient

| Generic assistant pattern                    | GROOT pattern                                                          |
|----------------------------------------------|------------------------------------------------------------------------|
| Prompt -> answer                             | Language -> mission -> answer / dataset / decision state               |
| Uses context for the current task            | Builds persistent organizational decision context                      |
| Meeting -> transcript / summary              | Meeting -> detected decision state and information gaps                |
| More data is generally helpful               | Acquire information according to mission need and decision sensitivity |
| Conversation ends when response is delivered | Decision can remain open, monitored and learnable                      |
| Memory is conversational context             | Memory is structured decision history tied to outcomes                 |

```

EXPLICIT MODE
User types or speaks:
"Should we switch supplier B?"

AMBIENT MODE
Meeting participants speak naturally:
"Supplier B is cheaper, but can they
handle Q4 volume?"

```

```

graph TD
    A[EXPLICIT MODE  
User types or speaks:  
"Should we switch supplier B?"] --> B[AMBIENT MODE  
Meeting participants speak naturally:  
"Supplier B is cheaper, but can they  
handle Q4 volume?"]
    A -.-> C[+]
    B -.-> C
    C --> D[NATURAL LANGUAGE UNDERSTANDING]
    D --> E[INTENT / EVENT DETECTION]
    E --> F[MISSION COMPILER]
    F --> G[+]
    G --> H[ANSWER / ANALYZE]
    G --> I[DECISION / DATA]
    H --> J[+]
    I --> J
    J --> K[CONTEXT ENGINE]
    K --> L[INTELLIGENCE OUTPUT]

```

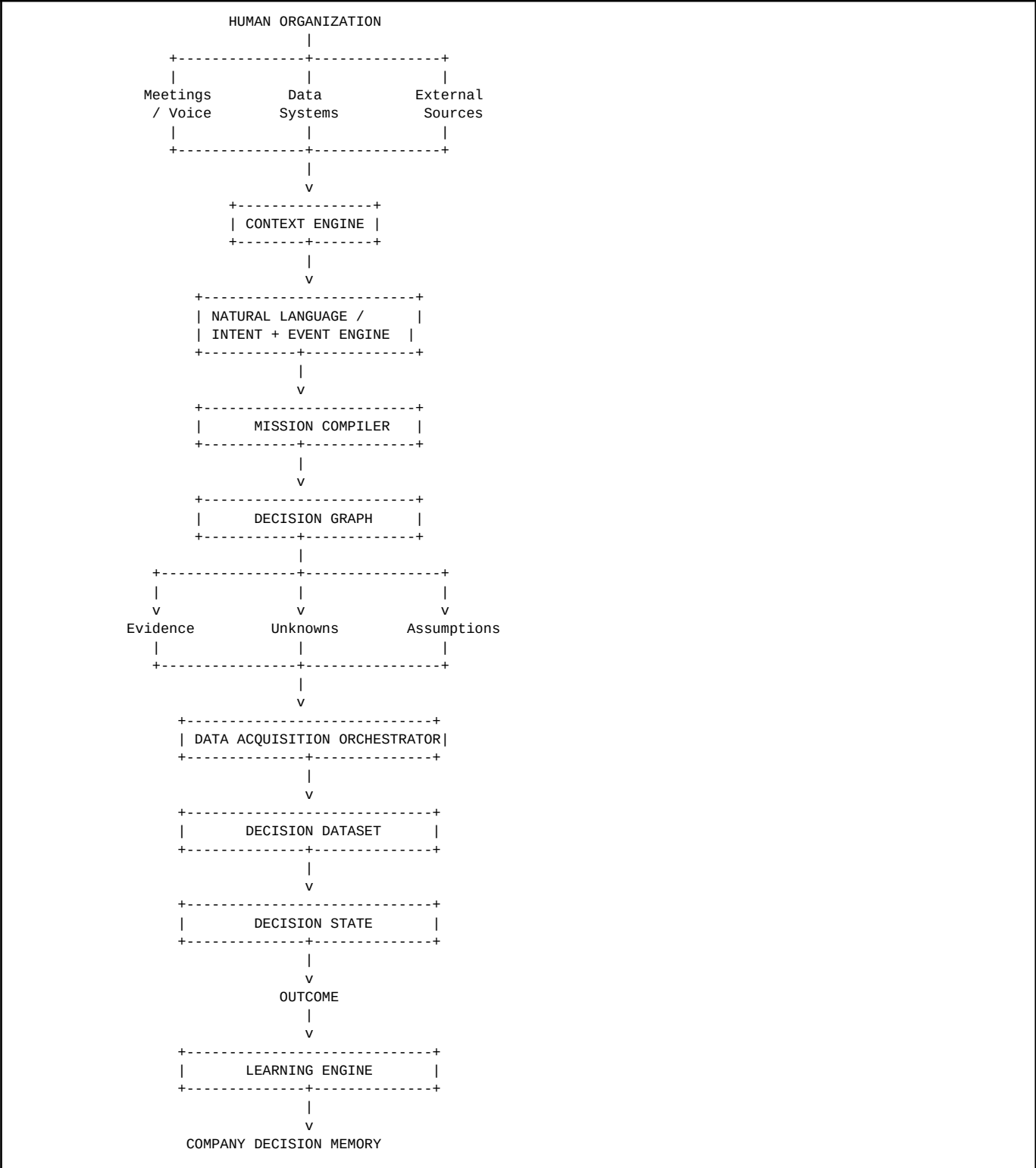
The flowchart illustrates the AI system architecture. It begins with two input modes: EXPLICIT MODE (User types or speaks: "Should we switch supplier B?") and AMBIENT MODE (Meeting participants speak naturally: "Supplier B is cheaper, but can they handle Q4 volume?"). These inputs feed into a central processing block (indicated by a dashed line with '+' at the ends). The output of this block is NATURAL LANGUAGE UNDERSTANDING, which leads to INTENT / EVENT DETECTION, then MISSION COMPILER. The MISSION COMPILER outputs feed into two parallel paths: ANSWER / ANALYZE and DECISION / DATA. These paths converge into a single block (indicated by a dashed line with '+' at the ends), which then feeds into the CONTEXT ENGINE. Finally, the CONTEXT ENGINE produces the INTELLIGENCE OUTPUT.

Voice is only the transport mechanism. The differentiated capability is that GROOT can recognize an implicit information requirement from ordinary human language. If a meeting reveals that a decision depends on a missing variable, GROOT can convert that observation into a mission even when nobody formally created a prompt.

| Request type     | Example                                          | GROOT output                                   |
|------------------|--------------------------------------------------|------------------------------------------------|
| Direct question  | “What were Q2 sales in Maharashtra?”             | Answer grounded in permitted data.             |
| Investigation    | “Why did sales decline?”                         | Hypothesis / evidence analysis.                |
| Decision         | “Should we switch suppliers?”                    | Decision graph + required information + state. |
| Data requirement | “Find manufacturers matching these constraints.” | Structured data mission + result dataset.      |

GROOT should not activate the full decision pipeline for every request. The mission compiler first determines what the user is actually asking, then invokes only the necessary downstream reasoning and acquisition components.

# GROOT architecture



## ARCHITECTURE PRINCIPLE

The components are modular: language understanding decides what mission is required; the mission compiler creates the structured task; the acquisition layer obtains the required information; decision state persists what matters; and the learning layer closes the outcome loop.

# From natural language to an executable intelligence mission

The mission compiler is the heart of the product. It translates a human request into a formal internal representation. This is the layer that lets GROOT support multiple kinds of requests without asking the user to know the underlying data model.

1. Parse the user language. Identify the object, desired outcome, constraints, time horizon and level of certainty required.
2. Classify the request: direct answer, analysis, decision support, or structured data acquisition.
3. Build a task representation. Translate the request into variables, entities, relationships, evidence requirements and output schema.
4. Map variables to available context. Identify which requirements can be satisfied from internal systems, documents, permitted external sources or conversation context.
5. Find information gaps. Distinguish between unavailable data, unknown variables, unresolved assumptions and conflicting evidence.
6. Plan acquisition. Retrieve only the information needed to complete the mission, using the organization's permitted tools and sources.
7. Construct the decision-ready result. This may be a direct answer, analysis, dataset, or living decision state.
8. Persist the useful structure. Save the decision state and relevant reasoning context when the task is consequential or ongoing.

## Decision Graph example

USER: "Should we move 30% of production from Supplier A to B?"

### DECISION

```

|
+-- Option A: Maintain allocation
+-- Option B: Shift 30%
|
+-- Critical variables
    +-- Unit cost
    +-- Lead-time distribution
    +-- Defect rate
    +-- MOQ
    +-- Transition cost
    +-- Peak capacity
    |
    +-- Known
    +-- Unknown
    +-- Assumed
    +-- Conflicting
    |
    +-- Required data mission(s)
    +-- Current decision state
  
```

### DESIGN PRINCIPLE

The system should optimize for completion of the user mission, not for the number of sources queried or the amount of data collected.

# When the prompt is spoken, not typed

With explicit participant consent and a visible Decision Room state, GROOT can process live conversation. The goal is not to replace meeting assistants. The goal is to detect when ordinary conversation creates an implicit information requirement.

MEETING CONVERSATION

"Supplier B is cheaper."

"I'm worried about Q4 capacity."

"We don't have recent capacity evidence."

"Let's not decide until we know."

GROOT INTERNAL STATE

Decision: Supplier allocation

Status: BLOCKED

Critical unknown: Peak capacity

Known: Unit-cost advantage

Concern: Capacity risk

Required mission: Investigate supplier capacity

GROOT RESPONSE (when enabled)

"I detected one decision-critical unknown: Supplier B's Q4 capacity. I can investigate it before the decision is finalized."

## What GROOT extracts from conversation

| Signal                   | Meaning inside GROOT                                |
|--------------------------|-----------------------------------------------------|
| Decision statement       | A choice is being proposed, evaluated or finalized. |
| Assumption               | A belief is being used as a premise.                |
| Objection / disagreement | A variable or interpretation may be contested.      |
| Unknown                  | A missing fact may block or change the decision.    |
| Commitment               | An agreed follow-up or information request emerges. |
| Decision change          | New evidence changes the previously known state.    |

## Why this is not just “meeting notes”

A conventional meeting tool primarily compresses speech into a summary. GROOT converts speech into an evolving computational state: what the organization believes, what it needs to know, what decision is open, and what information would move the state forward. This distinction is central to the product.

FEASIBILITY NOTE

Modern speech-to-text and meeting-recording systems make the input channel technically practical. OpenAI documents meeting transcription and multi-speaker recording, but also emphasizes consent and accuracy limitations. GROOT therefore treats consent, retention and permission boundaries as core implementation requirements, not optional polish. [4]

# GROOT learns the organization through decisions and outcomes

“Self-learning” should not mean that the base language model is silently retrained after every interaction. The practical design is a persistent organizational memory that stores decision structures and outcomes, then uses those observations to update future decision workflows, relevant variables and information priorities.

```

PAST DECISION
-> Inputs known at the time
-> Assumptions
-> Information missing
-> Decision made
-> Predicted result

REAL WORLD OUTCOME
-> What actually happened

LEARNING ENGINE
-> Which variables were predictive?
-> Which assumptions failed?
-> Which evidence paths were useful?
-> Which unknowns should become standard inputs?

FUTURE DECISION
-> Updated decision model for this organization
  
```

## Decision Memory

| Stored element   | Purpose                                                    |
|------------------|------------------------------------------------------------|
| Decision context | What was being decided and under what constraints.         |
| Variables        | Which measurable factors were used or considered.          |
| Evidence         | Where each important input came from.                      |
| Assumptions      | What the organization believed when the decision was made. |
| Decision outcome | What choice was made.                                      |
| Observed outcome | What happened later.                                       |
| Learning delta   | What GROOT should change in a similar future mission.      |

## What the system can learn

- Repeated decision variables that consistently influence outcomes.
- Information that decision-makers repeatedly request before committing.
- Assumptions that repeatedly fail or require revision.
- Sources or internal systems that are repeatedly useful for particular decision variables.
- Patterns between the organization’s stated reasoning and its actual decisions and outcomes.

### TECHNICAL POSITION

The moat is not a proprietary foundation model. The moat is the accumulated structured record of how a specific organization frames decisions, which information it needs, what it decides, and what actually happens afterward.

# How GROOT directly satisfies the problem statement

The central Track 01 requirement is natural-language understanding of data requirements and dynamic workflow execution. GROOT treats that requirement as the product center: the user does not need to provide a pre-defined schema or collection recipe. The system derives the structure from ordinary language.

| Track 01 requirement                                  | GROOT implementation                                                                                                 |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Understand requirements from natural-language prompts | Mission compiler converts plain language into objectives, entities, variables, constraints and outputs.              |
| Dynamically design data-collection workflows          | Workflow is derived after the system determines what information is actually needed.                                 |
| Execute across multiple permitted sources             | Context engine + acquisition orchestrator work across connected internal and external sources subject to permission. |
| Clean / structure / validate information              | Mission-specific decision datasets normalize the information required for the current task.                          |
| Traceable / explainable outputs                       | Decision state retains source lineage and the reasoning structure behind important inputs.                           |
| Monitor / manage collection tasks                     | Open missions and unresolved variables remain part of the decision state until resolved or closed.                   |
| Maintain workflow / dataset history                   | GROOT maintains persistent decision and outcome memory rather than discarding context after the response.            |

## Why the interpretation is differentiated

A common interpretation of Track 01 is “natural language -> schema -> scrape / query -> table.” GROOT interprets it as “natural language -> determine the information mission -> determine what must be known -> acquire only what is needed -> produce the answer or decision state -> learn from the outcome.” This moves the center of gravity from collection to intelligence orchestration.

| SCRAPING IS NOT THE PRODUCT                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| External web collection can be one acquisition mechanism when permitted and useful. It is deliberately not the defining product abstraction. GROOT is valuable even when the required information already exists inside an ERP, CRM, document store, meeting transcript or connected business system. |

## Research basis

The design uses established ideas rather than claiming to invent decision theory itself. Value-of-information research provides a formal basis for prioritizing which additional information is worth collecting because it can reduce decision uncertainty or expected loss. GROOT operationalizes that principle as part of an AI mission compiler. [5]

# Is GROOT actually buildable? Yes - as a staged system.

The complete vision is ambitious, but the underlying building blocks are individually available: speech-to-text, LLM reasoning, retrieval, structured extraction, tool calling, event-driven workflows, databases, permissions, and APIs. The build challenge is systems integration and the definition of the decision/mission representation, not the invention of a new foundation model.

## Recommended MVP boundary

| Build in MVP              | Purpose                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------|
| Natural-language prompt   | Primary interaction and Track 01 compliance.                                                  |
| Mission compiler          | Turns prompts into structured missions.                                                       |
| Connected data sources    | Start with a small set: uploaded files + one business data source + approved external source. |
| Decision graph            | Represent variables, evidence, unknowns and assumptions for decision requests.                |
| Decision Room voice input | Live transcription and event extraction for one meeting flow.                                 |
| Decision state            | Persist open / resolved / blocked states.                                                     |
| Basic outcome memory      | Store decision + outcome pairs for later replay.                                              |

## Not required for a convincing prototype

- A proprietary foundation model.
- Dozens of enterprise connectors.
- Continuous model retraining.
- Large-scale web scraping infrastructure.
- A complex analytics dashboard.

## Reference implementation stack

|                                                                                                                                                                                                                                                                                    |                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Interface:                                                                                                                                                                                                                                                                         | Web app + microphone / meeting capture         |
| Speech:                                                                                                                                                                                                                                                                            | Streaming speech-to-text                       |
| Reasoning:                                                                                                                                                                                                                                                                         | LLM with tool calling / structured output      |
| Mission model:                                                                                                                                                                                                                                                                     | JSON schema + decision graph representation    |
| Orchestration:                                                                                                                                                                                                                                                                     | Python service + task queue / workflow runner  |
| Data:                                                                                                                                                                                                                                                                              | PostgreSQL + object storage                    |
| Retrieval:                                                                                                                                                                                                                                                                         | Metadata index + embeddings where useful       |
| Connectors:                                                                                                                                                                                                                                                                        | API / MCP / file adapters                      |
| Learning memory:                                                                                                                                                                                                                                                                   | Decision + outcome tables / graph layer        |
| Security:                                                                                                                                                                                                                                                                          | Tenant isolation + permission-aware connectors |
| Evaluation:                                                                                                                                                                                                                                                                        | Golden missions + decision replay tests        |
| <b>BUILDABILITY VERDICT</b>                                                                                                                                                                                                                                                        |                                                |
| A strong competition MVP is technically realistic. The difficult part is not making an LLM answer a question; it is making the mission representation, decision state and outcome loop reliable enough that the system behaves consistently across different prompts and meetings. |                                                |

# From hackathon prototype to startup-grade system

The roadmap should be staged so that each phase proves a distinct architectural claim. The competition build should demonstrate the core loop end-to-end rather than attempt every enterprise integration.

| Phase                             | Objective                                                                    | Proof                                                                      |
|-----------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Phase 1 - Prompt to mission       | Build natural-language compiler and structured mission object.               | User prompt reliably becomes an executable task specification.             |
| Phase 2 - Context and acquisition | Connect files, one internal source and one approved external source.         | Mission can gather and normalize required information.                     |
| Phase 3 - Decision graph          | Represent assumptions, unknowns and evidence.                                | Decision request produces a transparent state rather than a single answer. |
| Phase 4 - Decision Room           | Add live speech ingestion and event detection.                               | Spoken discussion can create or update the same mission/decision state.    |
| Phase 5 - Memory                  | Persist decisions and outcomes.                                              | Past decisions can be replayed and compared with actual outcomes.          |
| Phase 6 - Learning loop           | Update organization-specific variable and workflow priorities from outcomes. | Similar future missions use learned organizational context.                |

## Competition demo flow

1. User types: "Should we move 30% of production from Supplier A to B?"
2. GROOT identifies a decision and builds the critical-variable graph.
3. GROOT shows known vs unknown information.
4. User enters a Decision Room and says:  
"The concern is whether B can handle Q4 volume."
5. GROOT detects the new decision-critical unknown.
6. GROOT asks permission to investigate / calls permitted sources.
7. GROOT returns the decision-ready result and updates the decision state.
8. The decision is saved.
9. A simulated later outcome is introduced.
10. GROOT replays the decision and shows the learning delta.

## Evaluation metrics

- Mission compilation accuracy: does GROOT infer the intended task and constraints?
- Required-variable recall: does it identify the decision-critical variables?
- Acquisition efficiency: does it obtain the needed information without unnecessary work?
- Decision-state fidelity: can the system reconstruct what was known, unknown and assumed?
- Outcome learning: does a prior outcome change the next similar mission in a measurable way?

## STARTUP PATH

Begin with organizations where decisions are repeated, cross-functional and data-fragmented. The product can expand horizontally once the mission compiler, decision state and learning loop are reliable enough to become reusable infrastructure.

# Why GROOT is more than a normal chatbot agent

| Capability                                             | ChatGPT / Claude style assistant | GROOT                          |
|--------------------------------------------------------|----------------------------------|--------------------------------|
| Natural-language questions                             | Yes                              | Yes - core interface           |
| Research / connected data                              | Yes                              | Yes - as mission execution     |
| Meeting transcription                                  | Increasingly available           | Yes - as ambient mission input |
| Persistent decision object                             | Not the core abstraction         | Core abstraction               |
| Implicit information requirement from meeting language | Possible with tooling            | Core workflow                  |
| Decision graph: assumptions / unknowns / evidence      | Can be constructed ad hoc        | Persisted system state         |
| Outcome-linked decision memory                         | Not the core abstraction         | Core learning loop             |
| Organization-specific decision model                   | Requires custom setup and usage  | Designed as product state      |

The comparison must be framed carefully. OpenAI and Anthropic are rapidly adding research, connectors, enterprise context and meeting capabilities, while Microsoft is building an enterprise intelligence layer that reasons over organizational work context. [1][2][3][4] Therefore the defensible claim is not “existing assistants cannot do these things.” The claim is that GROOT makes a different system-level abstraction the product: the persistent intelligence mission and decision state that sit between natural language and organizational action.

## The final product thesis

|                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CORE</b>                                                                                                                                                      |
| Natural language is the primary interface. The user states what they want to know, investigate, compare, collect or decide.                                      |
| <b>INPUTS</b>                                                                                                                                                    |
| GROOT can receive either an explicit prompt or, with consent, natural spoken language from a Decision Room.                                                      |
| <b>ENGINE</b>                                                                                                                                                    |
| GROOT compiles that language into a mission, identifies what information is required, orchestrates acquisition and builds a structured result or decision state. |
| <b>MEMORY</b>                                                                                                                                                    |
| The system stores decisions and outcomes, creating an organization-specific decision memory that improves future missions.                                       |

## Final pitch

**“Every AI assistant waits for you to ask a question. GROOT can understand the question you explicitly ask - or recognize the question your organization is implicitly trying to answer. It converts natural language into intelligence missions, carries those missions through the organization’s information, and learns from the decisions that follow.”**

## Research references

[1] OpenAI Deep Research: <https://help.openai.com/en/articles/10500283-deep-research-in-chatgpt> | [2] Anthropic Enterprise / Connectors: <https://support.anthropic.com/en/articles/9797531-what-is-the-enterprise-plan> ; [3] Microsoft Work IQ APIs: <https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/work-iq-api-overview> | [4] OpenAI ChatGPT Record: <https://help.openai.com/en/articles/11487532-chatgpt-record> | [5] Value of Information review: <https://www.annualreviews.org/content/journals/10.1146/annurev-statistics-040120-010730>

Document status: Final concept definition for Track 01. Product scope is intentionally limited to the finalized architecture and features described in this paper.